

COMP1008 Part IV: Search Techniques

1. Define the problem: Search Tree
 2. Search for solutions:
 - Blind Searches (BFS, DFS, UCS)
 - A* (heuristic search)
-
- Game Playing
 - MiniMax
 - Alpha-beta Pruning
 - Course summary

COMP1008 - Fundamentals of AI

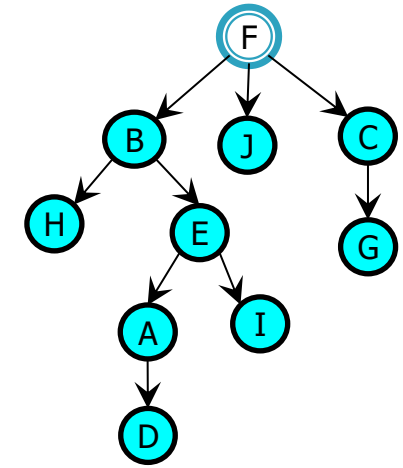
Game Playing

Professor Rong Qu
rong.qu@nottingham.ac.uk



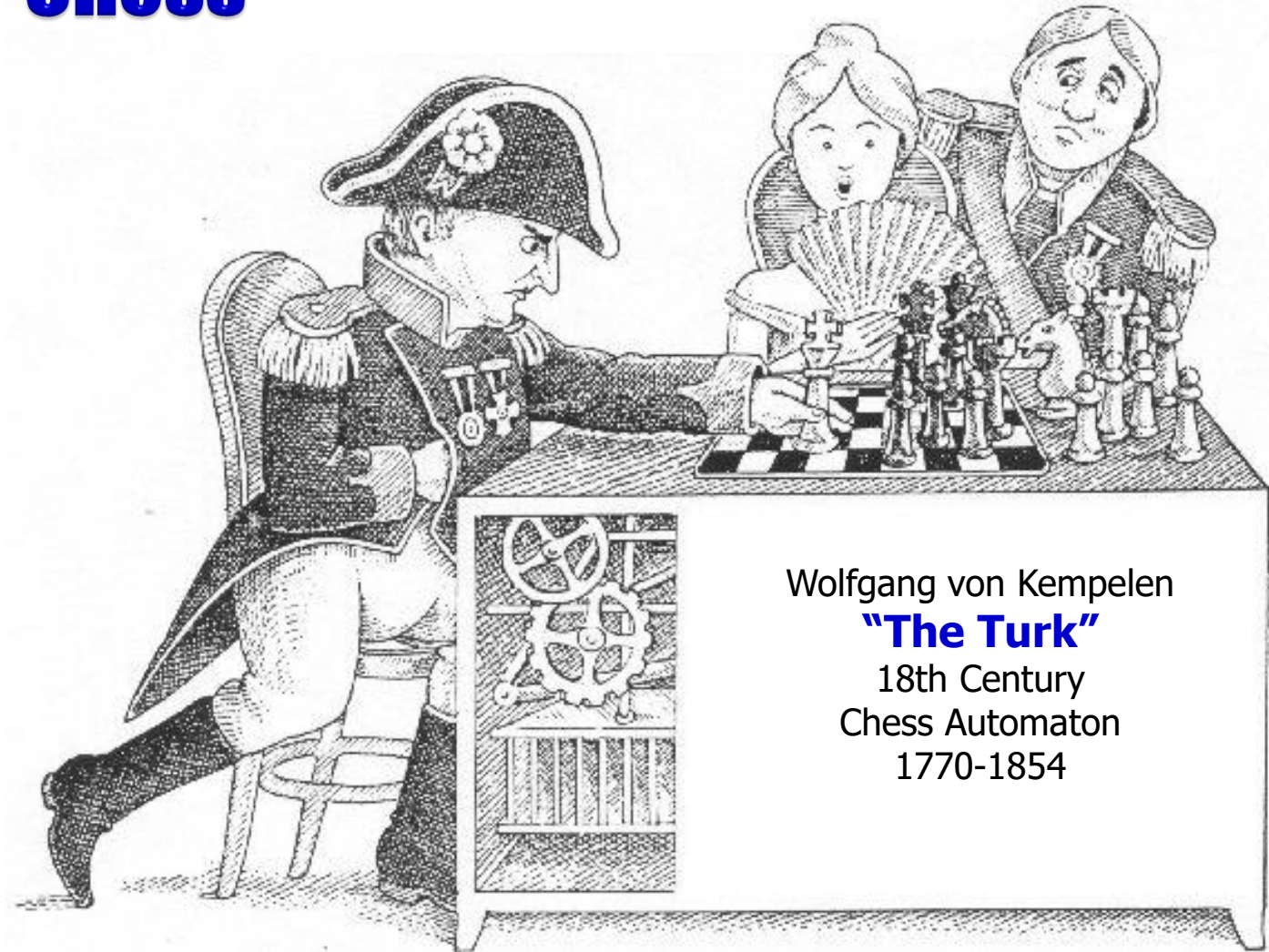
AI Game Playing

- ▶ Why study game playing in AI?
 - Games are **intelligent** activities
 - Do not require large amounts of **knowledge**
 - It is very easy to **measure** success or failure, with **simple rules**
 - Solvable by straightforward **search** from the starting state to a winning position
 - Tree search (**MiniMax**, **Alpha-beta Pruning**)

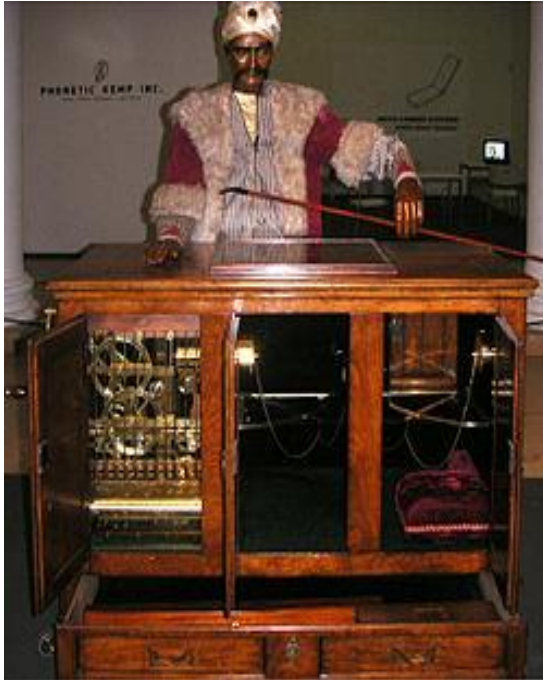


AI Milestones	COMP1008	CS Courses
Deep Blue	KRR + Alpha-beta Pruning	COMP3008 Symbolic AI
AlphaGo	Deep Learning + Alpha-beta	COMP3009 Machine Learning

AI Game Playing – Chess



AI Game Playing – Chess



AI Game Playing – Chess

No computer can play even an amateur-level game of chess
American philosopher Hubert Dreyfus, 1960's

A computer would be chess champion in 10 years
AI pioneers Newell and Simon, 1957

- ▶ **May 11th, 1997** - Gary Kasparov vs. Deep Blue, IBM Research
 - Six match game
 - 2:1 – Deep Blue vs. Kasparov
 - Three draws



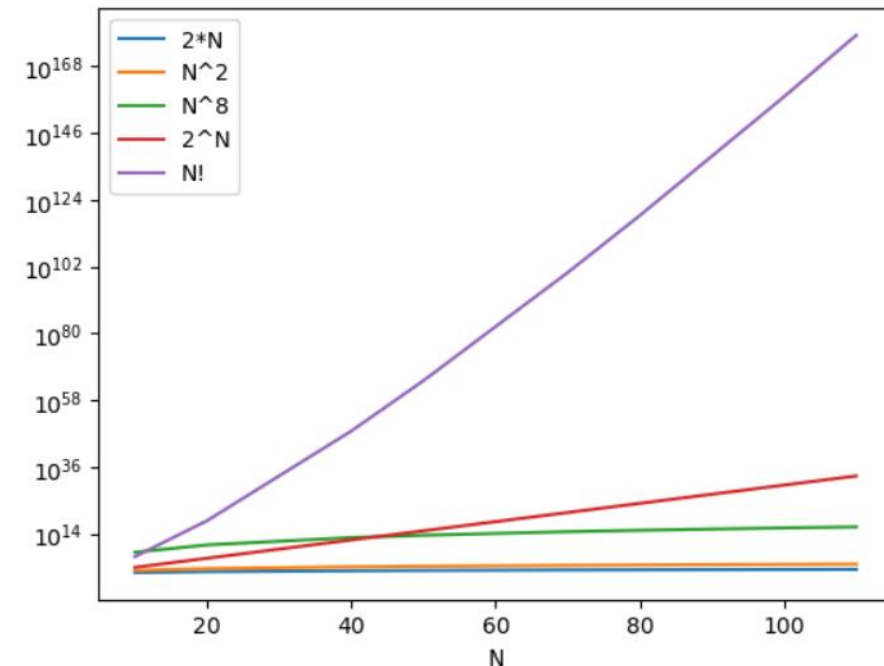
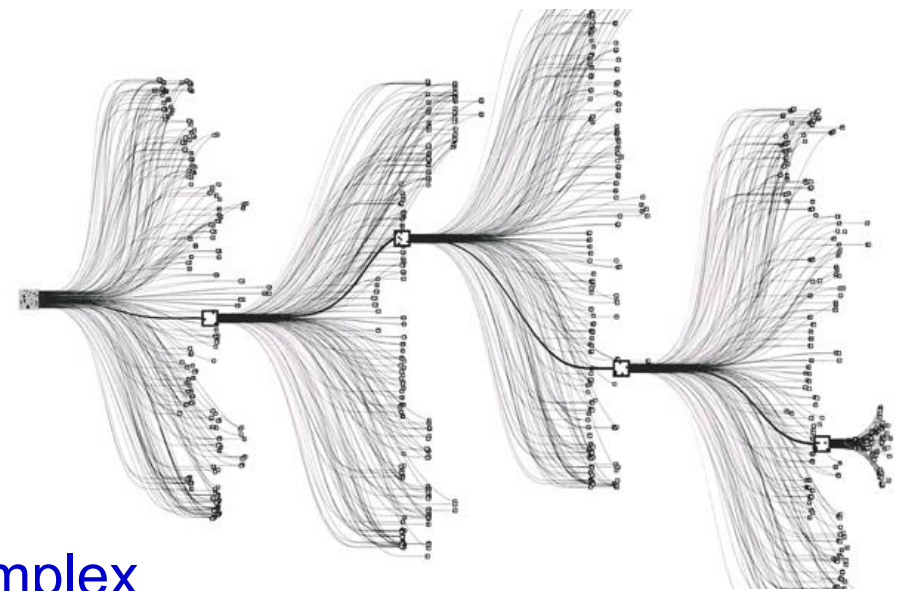
AI Game Playing – Go

- ▶ **AlphaGo** – Google DeepMind
 - Oct 2015: **first computer Go program** beat professional human Go player
 - Mar 2016: 4:1 beat 9-dan human player Lee Sedol
 - 29 Dec 2016 – 4 Jan 2017: 60:0 “**Master**” at Eweiqi
 - 29-31 Dec: won top players up to 9-dan, 41:0
 - 3 Jan: Jie Ke 9-dan
 - 4 Jan: “Thank you, Master Nie”
 - Dec 2017: **AlphaZero**



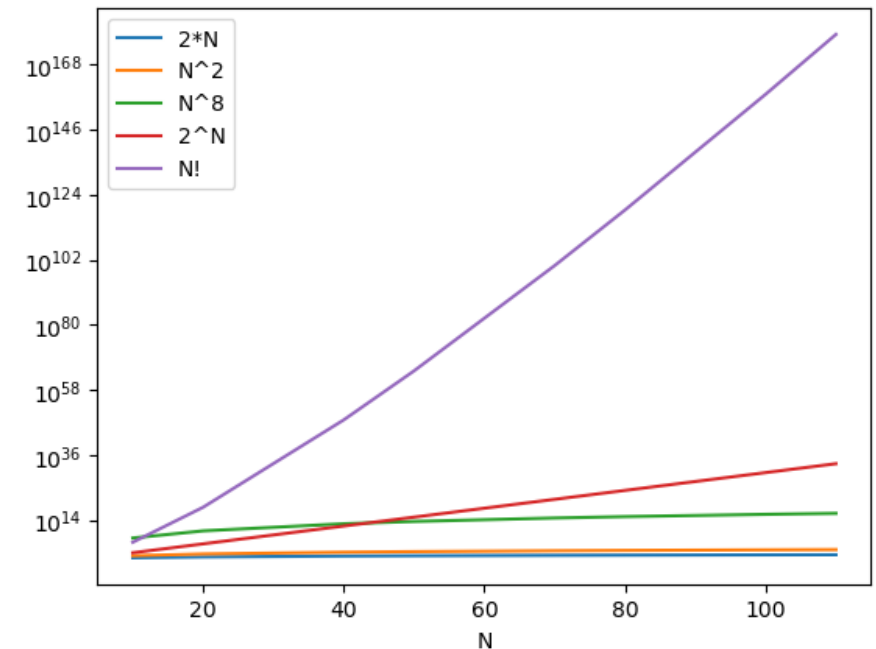
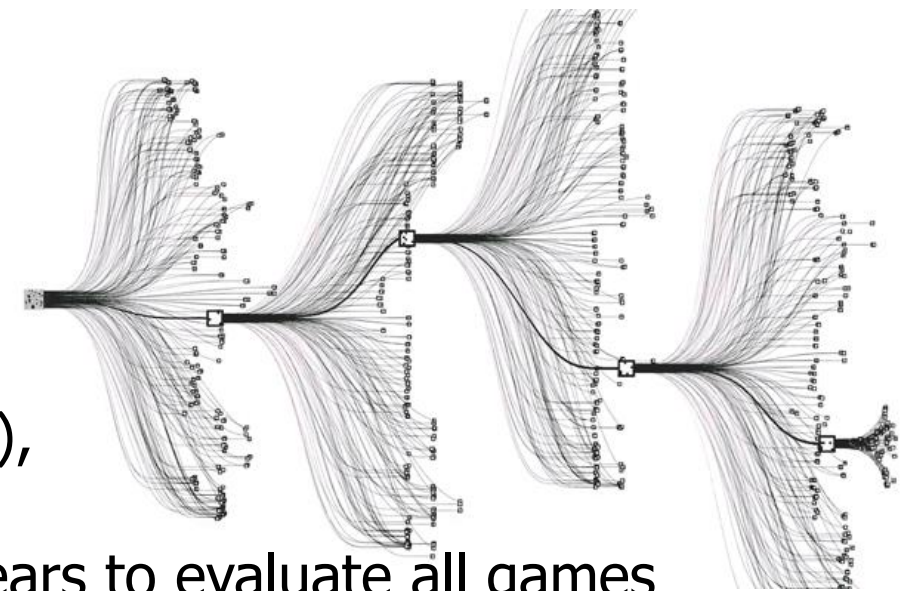
AI Game Playing – Go

- ▶ Significant challenge to CS and AI
 - **Larger** board (five times the chess board)
 - Not much helped by fast computation
 - New piece appears every move, getting more **complex**
 - A material advantage may just mean **short-term gain**
 - High degree of **pattern recognition** involved in human capacity

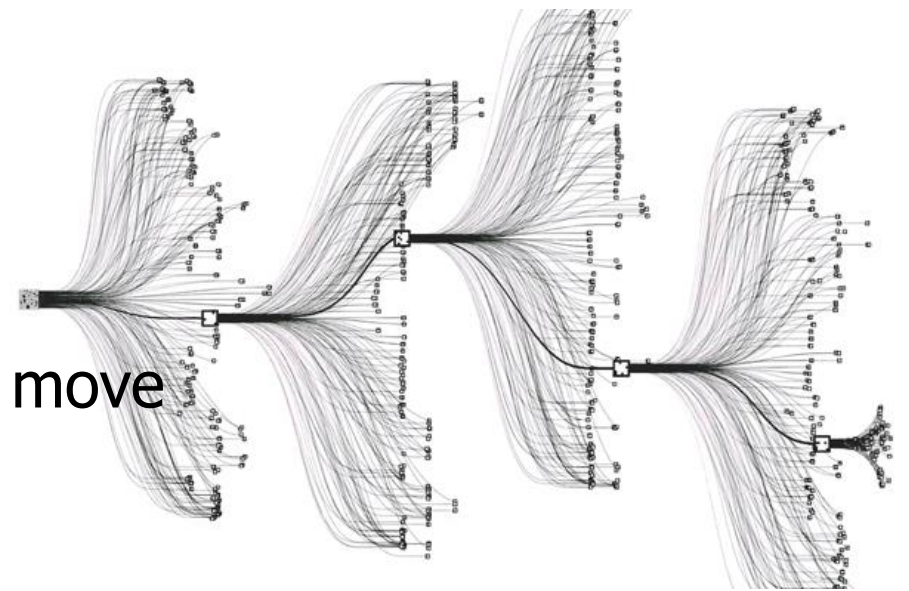


AI Game Playing – Go

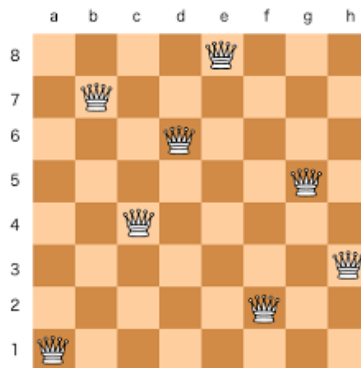
- **Claude Shannon** (1949)
 - 10^{120} unique states (i.e. avg 40 moves in a game), larger than the no. of atoms in the universe
 - 200M positions/s: Deep Blue would need 10^{100} years to evaluate all games
 - The universe is only about 10^{10} years old
 - Search trees grow **exponentially**
 - Which move (**node to expand**) next?



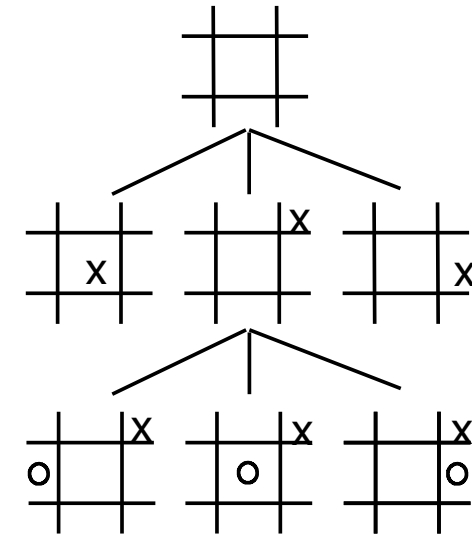
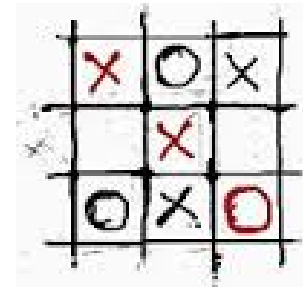
AI Game Playing - MiniMax



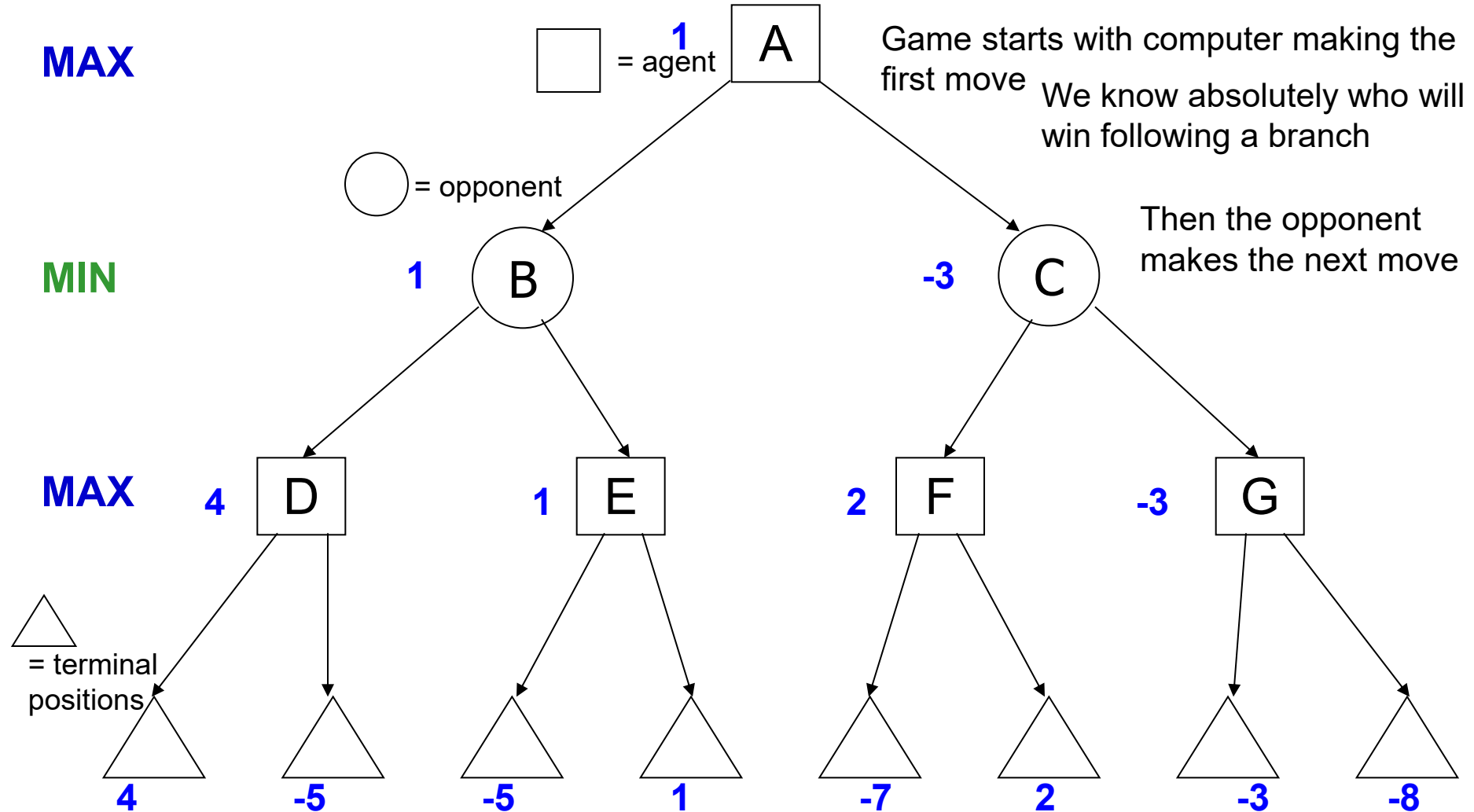
- ▶ An opponent tries to **prevent** your win at every move
- ▶ 1944 - John von Neumann
 - A tree search method (**MiniMax**), backtrack $\sim$ BFS
 - **Maximise** your position whilst **minimising** your opponent's
 - **Utility function**: measure how good a position is



5	4	
6	1	8
7	3	2



Assume we can generate the full search tree
 The idea is computer wants to force the opponent to lose, and
 Of course for larger problem it's not possible to draw the entire tree
 maximise its own chance of winning



Assume a positive game value means the computer wins, and a negative value means the opponent wins. Now the computer can decide who win the game

- whose turn it is, and
- whether they are trying to maximise or minimise at the node

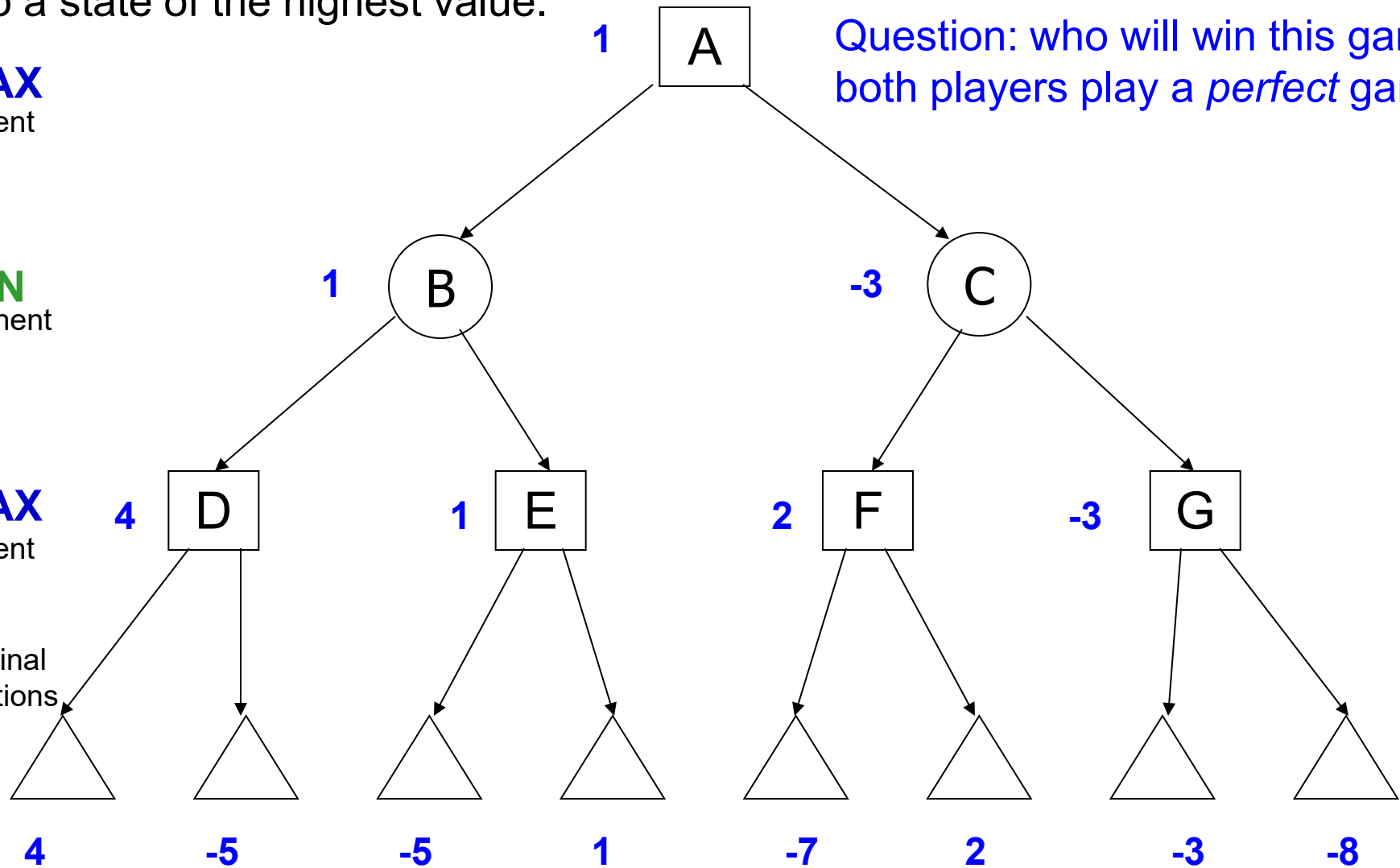
Now the computer is able to play a perfect game. At each move it'll move to a state of the highest value.

MAX
agent

MIN
opponent

MAX
agent

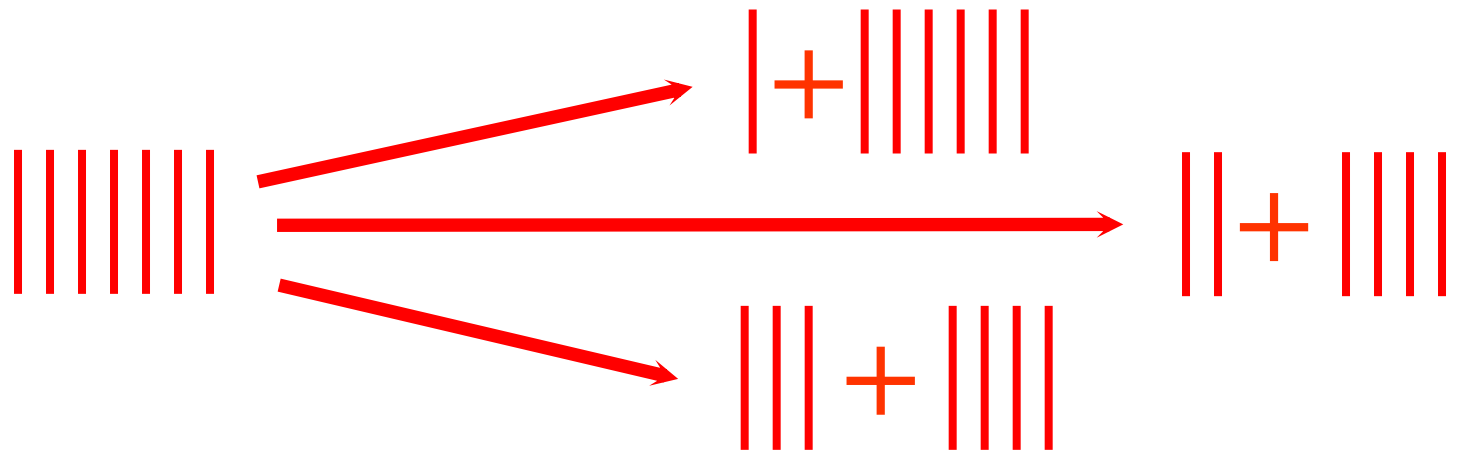
terminal
positions

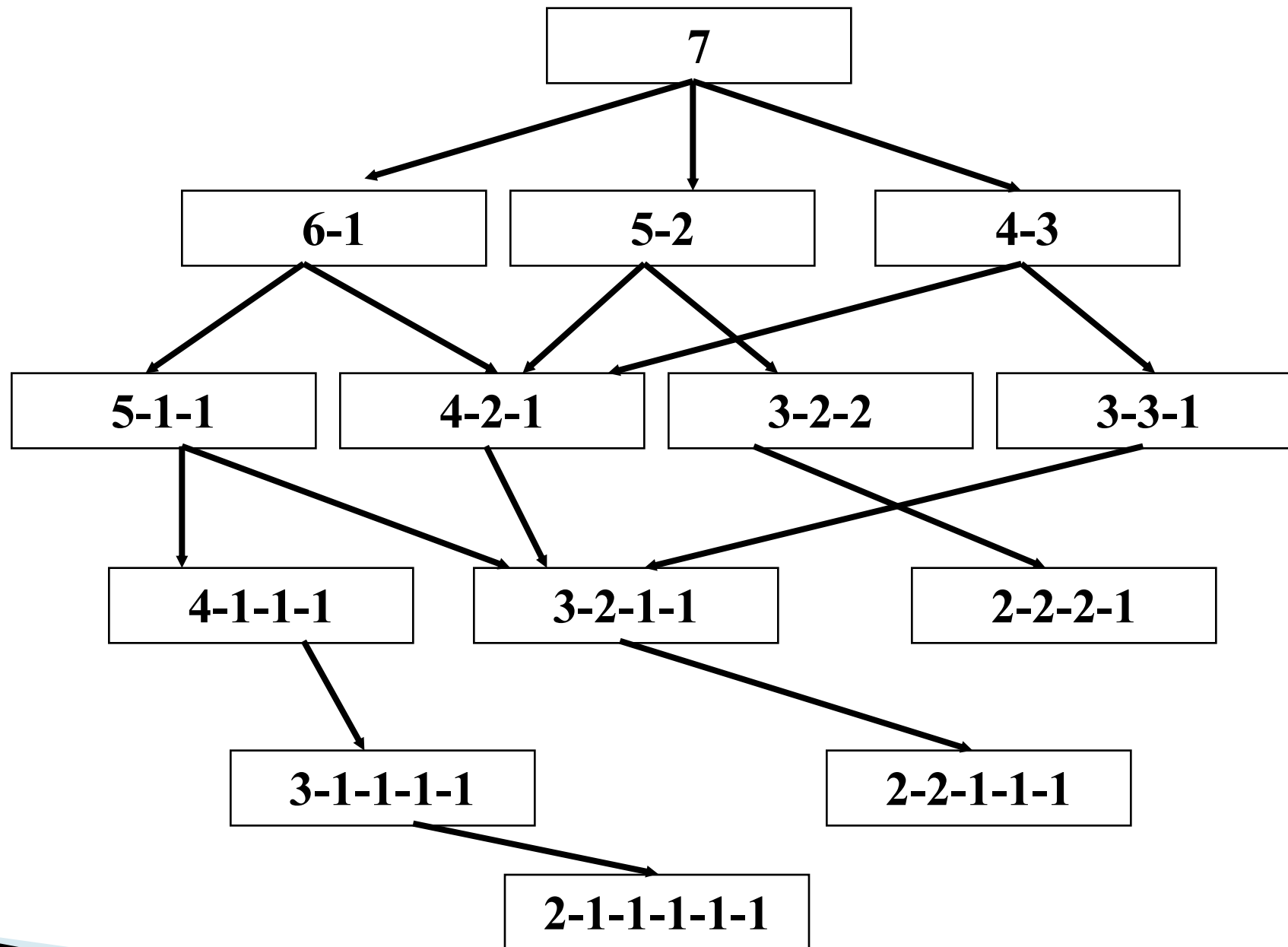


AI Game Playing - MiniMax

▶ Nim

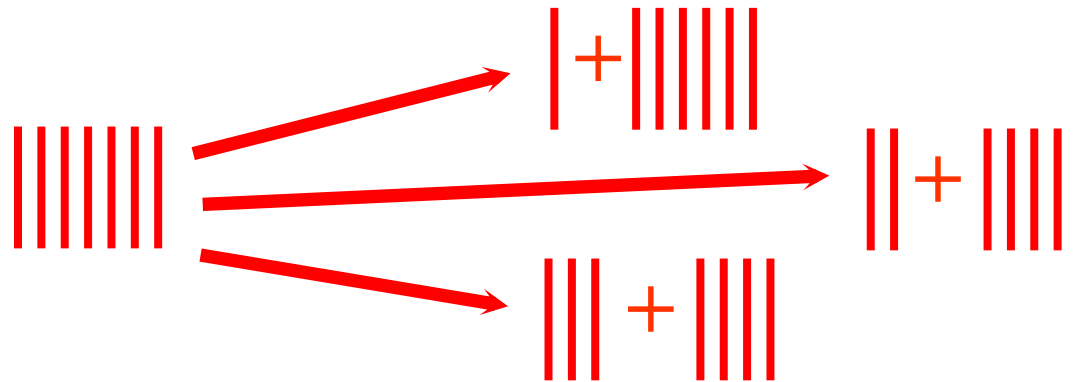
- Start with a pile of tokens, at each move the player must **divide** the tokens into two **non-empty, non-equal** piles
- The player who can't make the next move loses the game
- Starting with 7 tokens, draw the complete search tree

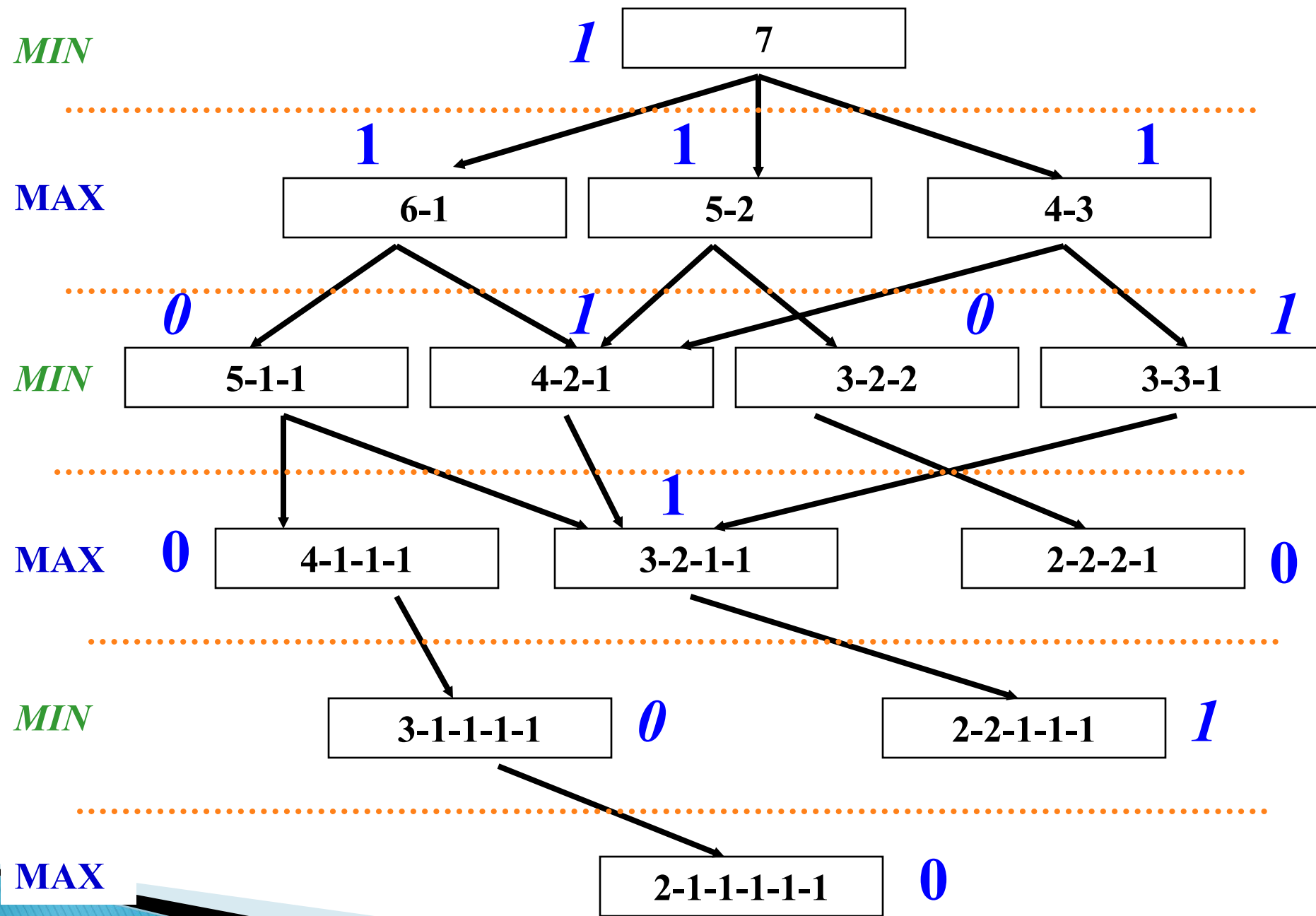




AI Game Playing - MiniMax

- ▶ **MAX**: the best move for MAX
 - The node with the **highest utility**, i.e. maximum of child nodes in the tree
- ▶ **MIN**: the best move for MIN (the worst for MAX)
 - The node with the **lowest utility** i.e. minimum of its child nodes in the tree
- ▶ Assume that a **Utility function**: for MAX (larger values are better for "MAX")
 - **0** = a win for **MIN**
 - **1** = a win for **MAX**





AI Game Playing – Past Exam Question

- ▶ A variant of Nim starts with a single stack of 6 tokens. At each move a player removes one, two or three tokens, leaving a non-empty stack. A player who is unable to move loses the game. **Min plays first.**

Draw the complete search tree. Merge the child states which are the same on each level of the tree into one single state.

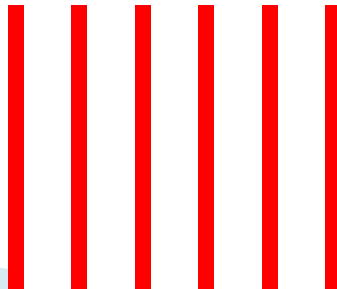
(6 marks)

- ▶ Apply the Minimax algorithm to the search tree, i.e. assign utility values to all states. A utility value of 0 is assigned to a terminal state if Min wins; otherwise, 1 is assigned to a state if Max wins the game.

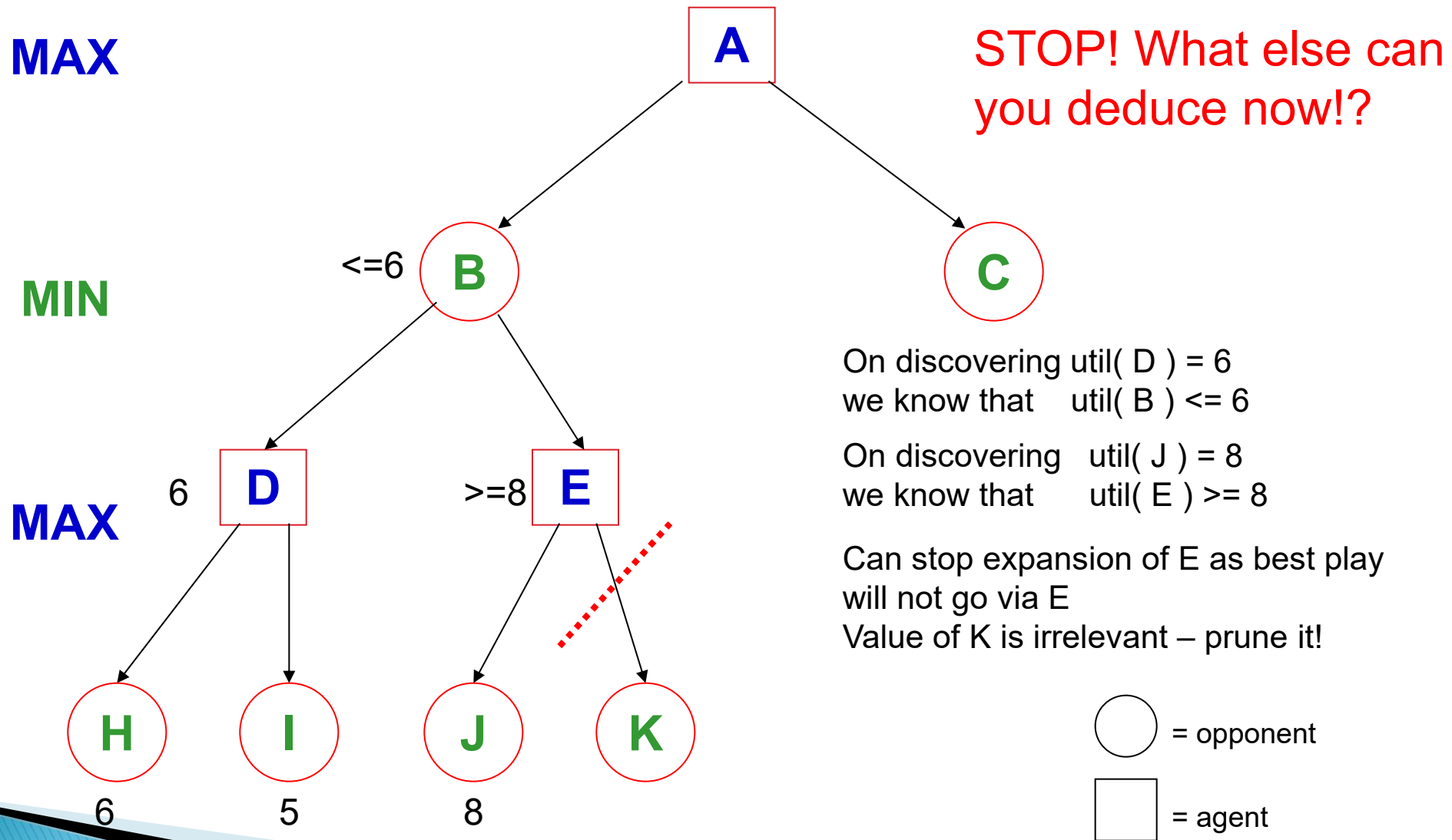
(4 marks)

- ▶ If both Min and Max play a perfect game, who will win the game?
In total there are how many winning game(s)/path(s) for this player?

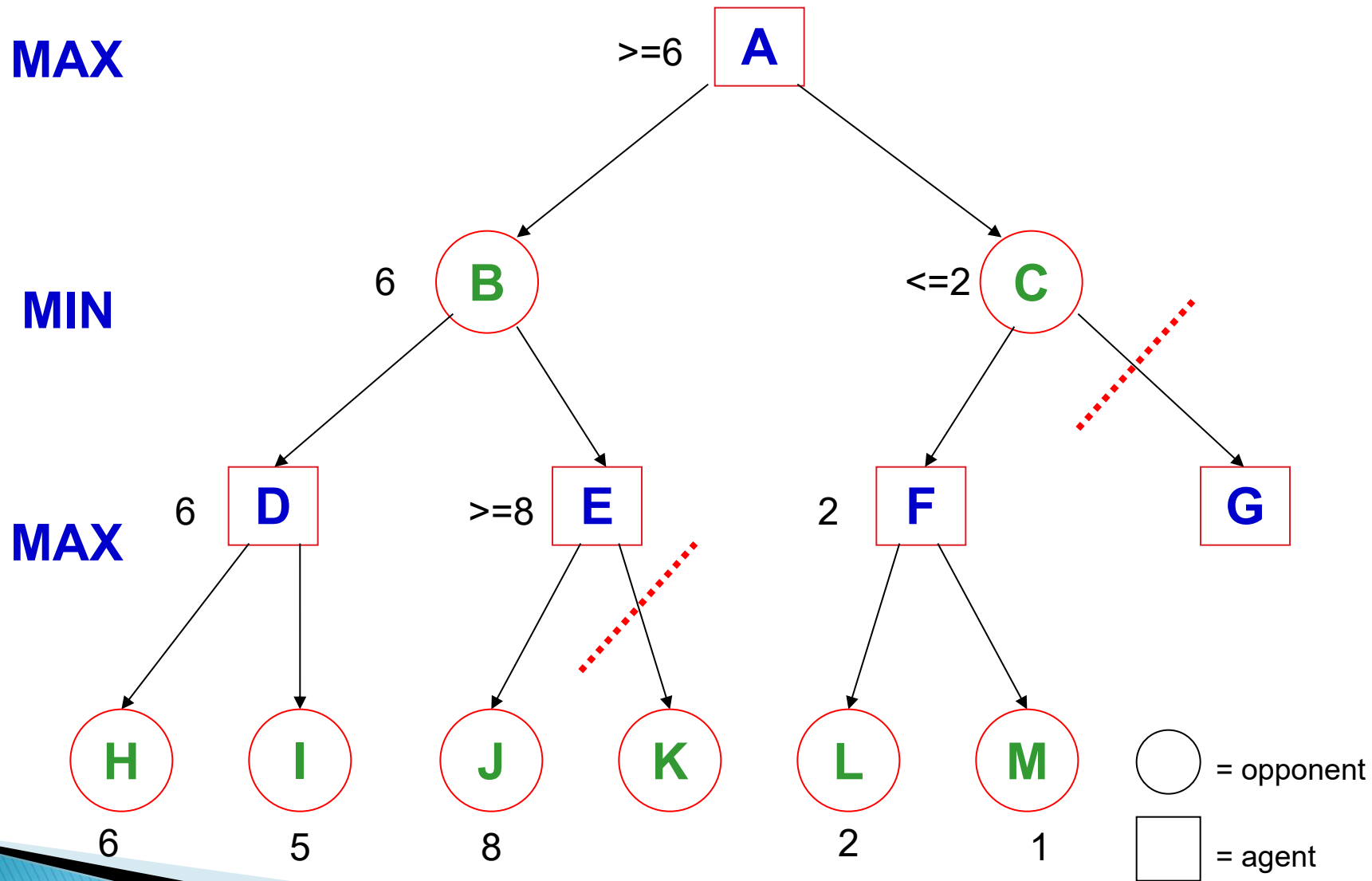
(2 marks)



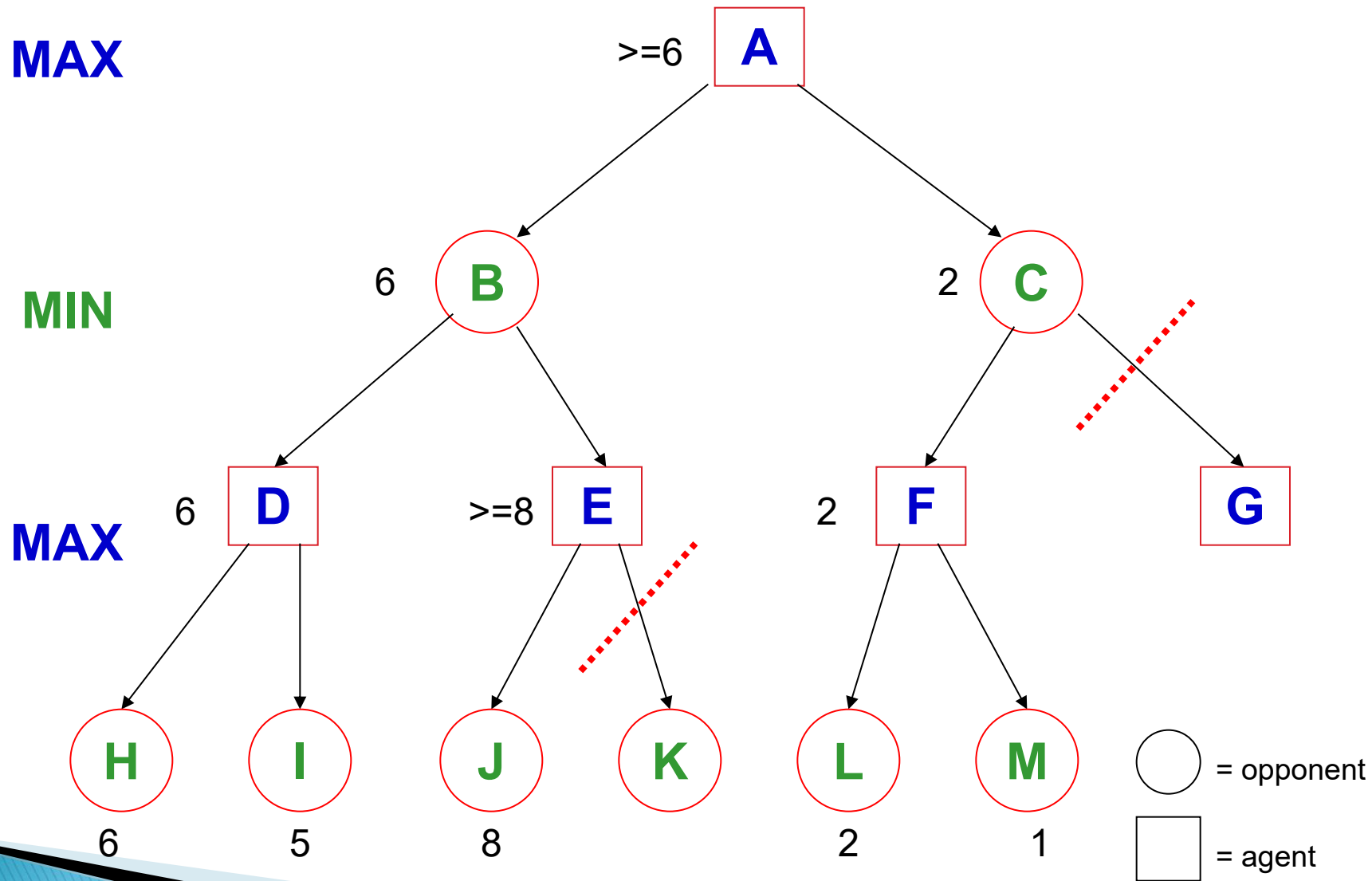
AI Game Playing – Alpha-Beta Pruning



AI Game Playing – Alpha-Beta Pruning



AI Game Playing – Alpha-Beta Pruning

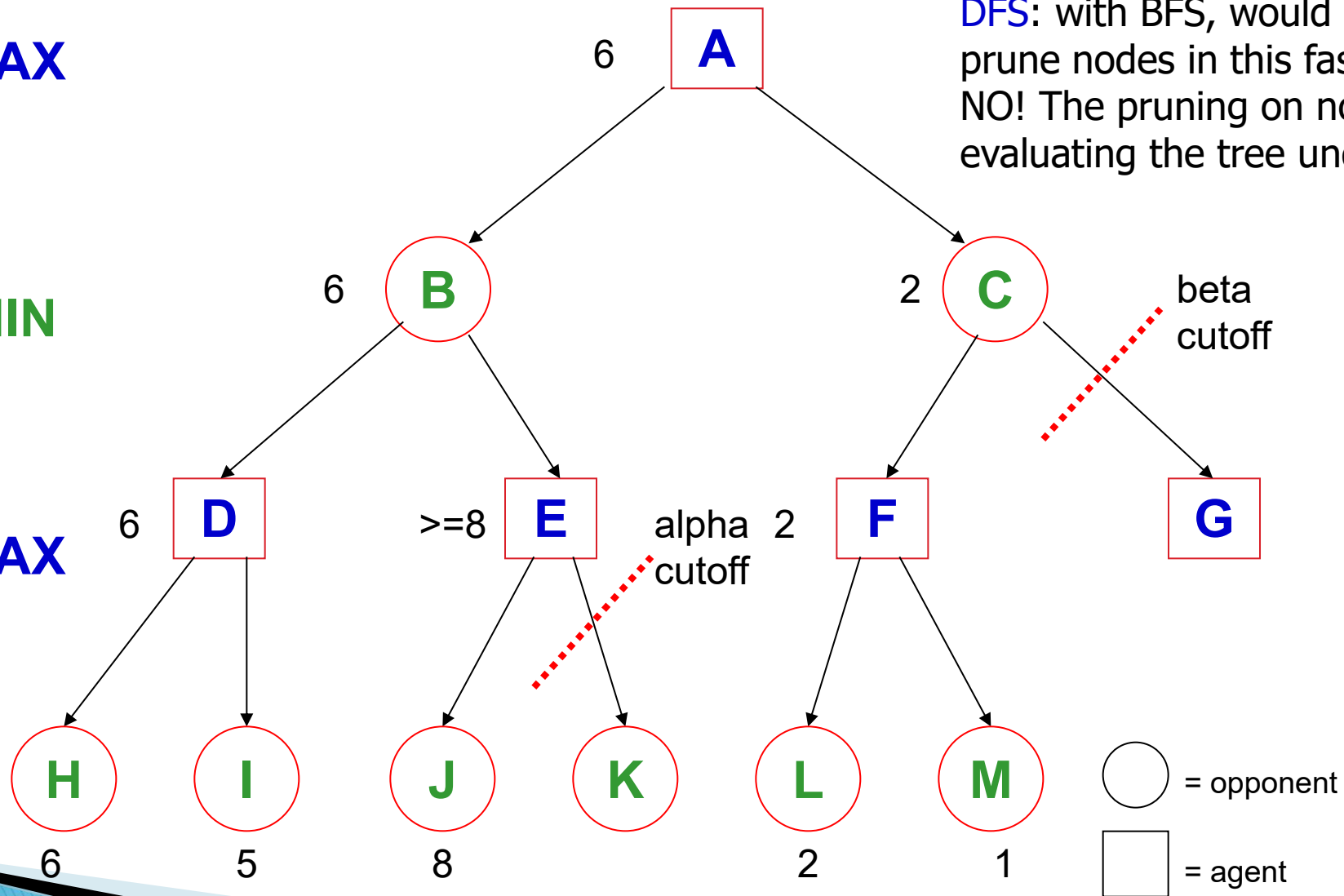


AI Game Playing – Alpha-Beta Pruning

MAX

MIN

MAX



DFS: with BFS, would you still be able to prune nodes in this fashion?
NO! The pruning on node D is made by evaluating the tree underneath D

AI Game Playing

1. Define the game: into **game tree**
 2. **Search** the huge tree: time-consuming
- ▶ **DeepBlue (Chess)**
 - Tree search + utility function: **MiniMax**
+ bounding strategies: **Alpha-beta pruning**
+ large **opening/ending boards**
 - ▶ **AlphaGo (Go)**
 - **MiniMax** + **Alpha-beta Pruning**
+ **Deep Learning**



AI Game Playing – AlphaGo

- ▶ How to reduce the number of explored nodes?
 - [Alpha-beta Pruning](#) based on [MiniMax](#) **estimate** of utility values (possibility of win) based on human game
- +
- [Machine learning](#) (2 X CNN deep learning)
 - Supervised Learning (images/patterns)
 - Reinforcement Learning



AI Game Playing – AlphaZero

▶ AlphaZero

- From scratch: No supervised learning, just one CNN on boards from self-playing
- Monte Carlo Tree Search: look-ahead by simulations

COMP1008

MiniMax, Alpha-beta

CS Courses

COMP4002 – Games
(more HCI than AI)



AI Game Playing – Summary

- ▶ **MiniMax**
 - How does it work?
 - Apply MiniMax for a given **new** game
- ▶ **AlphaBeta Pruning**
 - Built upon MiniMax
 - Where are cuts applied?
 - Utility values in the final / pruned tree

